Problem 3: Intern Hackathon: Group Travel Matcher

1. Background

Planning a solo trip can be lonely and expensive. Many travellers want to split costs, share experiences, and make friends on the way. Booking.com has millions of solo travellers searching trips—some want to create group plans and recruit others, some want to join existing groups.

Your task: build a matching platform where solo travellers can either **create a group trip post** (and wait for others to join) or **search for existing group posts** that match their interests. Once a group forms, members collaborate to search and book hotels, and coordinate on a shared page. Help them see: *Are we a good match? How much will it cost each of us?*

2. The Travellers

Scenario 1: Sarah (Group Creator)

Sarah plans a 5-night trip to Kyoto in April 2027. She creates a group post to recruit travel companions with shared interests (temples, street food) and a similar budget.

Scenario 2: Priya (Group Seeker)

Priya is also planning Kyoto in April 2027. She **searches for existing groups** that match her dates, destination, and interests. She finds Sarah's post and expresses interest.

Request Attribute	Sarah (Creator)	Priya (Seeker)
Home city	Bangalore (BLR)	Mumbai (BOM)

Request Attribute	Sarah (Creator)	Priya (Seeker)
Destination	Kyoto	Kyoto
Dates	10–15 Apr 2027 (5 nights)	9–16 Apr 2027 (7 nights, overlap: 5)
Interests	Temples, street food, photography, budget stays	Temples, street food, adventure
Budget	₹60,000 per person	₹62,000 per person
Gender	Female	Female
Gender preference	Female only (for safety)	Female only
Action Taken	**Creates a group post** with group name, proposed itinerary, # of people wanted (1–2 more), approx budget	**Searches for groups**, finds Sarah's post, clicks "Interested"

3. What You Build

A web application with a **backend** and a **frontend** that work together. Two distinct flows:

Flow 1: Group Creator

```
None
Sign Up → Create Group Post → Wait for interest → Approve members →
Common Group Page → Search hotels together → Cost split & book
```

Flow 2: Group Seeker

```
None
Sign Up → Search for groups → See matches → View group details → Express interest → (if approved) Join → Common Group Page → Search hotels together → Cost split & book
```

Everything comes from the data pack: 24 mock traveller profiles, 12 open group posts, interest tags, and hotels in 5 cities (Tokyo, Kyoto, Osaka, Dubai, Abu Dhabi) from 1 to 20 April 2027.

4. Technical Rules

Rule	Detail
Any stack	Any language, framework, and database.
Backend holds logic	Matching groups to travellers, ranking, approvals, cost calculation, and compatibility checks run in the backend.
Frontend uses backend API	The frontend gets all data from your backend API. The frontend does not read the data files directly.
State persistence	User profiles, group posts, memberships, and saved interests must persist after a page refresh and backend restart.
Dynamic requests	Judges will enter new travellers you have not seen: other dates, other cities, other interests, other budgets, all inside the data (5 cities, 1–20 April 2027). Your request forms must accept any valid values. Hard-coded results will fail.
No AI inside app	Your app must not call an AI or LLM model (cloud or local). All tools to help you write code are allowed.
Quick start	Start in 1–2 commands. Write the commands in your README.

Rule	Detail
Event development	Templates and open-source libraries are allowed. Feature code is written during the event. We check the Git history.

5. Inputs You Receive

Read DATA_DICTIONARY.md before you start. It explains every file and field, how the files connect, and gives worked examples. All prices are in INR. The data pack has these files:

File	What it is
data/problem-3/travellers.json	24 traveller profiles: name, age, gender, home city, interests, gender preference
data/problem-3/groups.json	12 open group posts: owner, members, city, dates, group size, budget, itinerary
data/problem-3/interests.json	16 interest tags
data/catalog/destinations.json	The 5 cities and their countries
data/catalog/hotels.json, rooms.json, availability.json	Hotels, room types and prices, and free rooms per night, for the Common Group Page

Group Matching Rules

Rule	Detail
Dates must overlap	The group and the seeker share at least 2 nights. Example: Sarah 10–15 Apr, Priya 9–16 Apr share the nights of 10, 11, 12, 13, and 14 Apr (5 nights) → match.
Same destination	Both travel to the same city. A country or a list of countries is not enough. Example: Kyoto and Kyoto → match. Kyoto and Osaka → no match.
Budget compatibility	The difference between the two budgets is at most 40% of the lower budget. Example: ₹60,000 and ₹62,000 → match. ₹60,000 and ₹84,000: the difference ₹24,000 is 40% of ₹60,000 → match. ₹60,000 and ₹85,000 → no match.
Shared interests	The seeker shares at least 2 interests with the group owner. Example: Sarah (temples, street food, photography, budget stays) and Priya (temples, street food, adventure) share 2 → match.
Gender preference	The seeker's preference allows the owner's gender, and the owner's preference allows the seeker's gender. Example: Sarah (female, female only) and Priya (female, female only) → match. A male seeker → no match.
Group size	Current members + the seeker is not more than the group size the owner asked for. Example: the owner wants 3 people in total and the group has 2 → matches. The group has 3 → no match.

6. Features

Must Have Features

ID	Feature	Acceptance Criteria
M1	Sign up flow	Users set up their profile by inputting details like name, gender, origin city, and personal interests. A basic identity tied to a name or active session is sufficient, so complex authentication via email or password is not needed. Profile details must stay saved even after refreshing the page.
M2	Group owner	Form to create a group post: destination, check-in/check-out dates, # of people wanted (1–4 total), proposed itinerary (text), group name, approx budget. The city is one of the 5 cities in the data, and the dates are inside 1–20 April 2027.
M3	Traveller search	Seeker enters home city, destination, dates (check-in and check-out), interests, budget, gender preference. The city is one of the 5 cities in the data, and the dates are inside 1–20 April 2027.
M4	See matches	The app shows a list of group posts that match the seeker's criteria: all 6 rules in section 5 (same city, dates overlap, budget, shared interests, gender preference, group size). Each match shows: group name, destination, dates, group members count, owner name.
M5	Accept or decline	Seekers can express interest for a group. The group owner sees pending requests and can approve/reject. The state is saved.
M6	Common Group Page	Group members search and compare hotels using destinations and dates. Results display hotel details

ID	Feature	Acceptance Criteria
		and per-member cost (price ÷ members count). Once the group agrees offline, clicking the Book CTA triggers a popup to confirm the booking. No actual booking creation is needed.

Good to Have Features

ID	Feature	Acceptance Criteria
G1	Interests filtering	On the group search results, seekers can filter by shared interests and gender composition.
G2	Group members profile pages	Seekers can visit each group member's profile page to see their interests, home city, age, and the groups they are in.

Stretch Features

ID	Feature	Acceptance Criteria
S1	Chat feature	Chat messages capability on the common group page for members to discuss plans.

---

7. What the Judges Look For

Evaluation Area	Weight
Working end-to-end feature (Must / Good / Stretch reached)	25%
Correctness on new requests and changes given by the judges at demo time	25%
Individual understanding (questions to each team member, and one small live code change)	20%
Product and user experience (is the match reason clear? Is group coordination intuitive?)	15%
Engineering quality (code structure, data model, README, Git history)	15%

Important: A match that breaks a rule (other city, fewer than 2 shared nights, budget difference more than 40%, fewer than 2 shared interests, gender mismatch, group full) is wrong, even if it looks good. Correct results matter more than the number of features.

---

8. Common Group Page: Search & Book Flow

Once a group is formed (creator approved seeker), all members access the **Common Group Page** to search and finalize bookings.

Key Flow Elements

1. **Search by Group Context:** Members search using group dates & destination
2. **Per-Person Cost Visibility:** Every option shows total AND per-person cost
3. **Save & Compare:** Members save multiple options, see group alignment
4. **Offline Consensus:** Group discusses offline (chat optional in Stretch), decides on one option
5. **Finalize Booking:** When consensus reached, ONE booking is created for all members
6. **Cost Split Updates:** Final booking updates the group's total trip cost & per-person split

9. The Demo (12 minutes)

Part	Time	Details
Live demo	4 min	Show the app on your laptop. **Scenario A:** One team member creates a group post (Sarah). **Scenario B:** Another team member signs up, searches for groups, finds Sarah's group, and expresses interest. Show the group owner approving the request. Then show the common group page with members searching hotels together and the cost split. At most 1 slide for the architecture.
Judge requests	3 min	The judges give you new requests and ask for changes. You do them live. (Example: "Add a new traveller with different dates and show matches again.")
Questions	4 min	The judges ask each team member questions about the code and design.
Changeover	1 min	The next team sets up.

Prepare your laptop before your slot: app running, sign-up forms empty, ready to start.

10. Submission

1. Push your code to your Git repository every hour.
2. At code freeze, upload a zip file to HackerEarth with your source code, `README.md`, and setup scripts. No `node_modules`, virtual environments, build output, or API keys.
3. The `README.md` says how to start the app, the architecture, what you finished, and known limitations.

11. FAQ

Can we use AI tools to write code or call an AI model in our app?

AI coding assistants are allowed to help write code (you must understand all submitted code), but your application cannot call AI/LLM models directly in the backend or frontend.

How should edge cases (no matches, full groups, search overflow) be handled?

Handle them gracefully in the UI. For no matches, suggest actions (extend dates, add interests). For full groups, disable interest buttons or waitlists. For large result sets, paginate or limit top results.

How do we define ranking and tie-breaking for matches?

You decide the weighted formula (e.g., shared interests, budget fit, date overlap) and tie-breaking rules (e.g., earliest date or larger group size). Explain your logic during the demo.

Can we store extra data beyond the catalog?

Yes. User profiles, group posts, memberships, saved interests, hotel search history, and bookings go in your database. Don't modify the original catalog files.

How do we handle timezones?

Read DATA_DICTIONARY.md. Dates are in the traveller's local timezone. Use the city's timezone from the catalog.

Which cities and dates can we use?

Only the cities and dates in the data: Tokyo, Kyoto, Osaka, Dubai, and Abu Dhabi, from 1 to 20 April 2027. For a city or date outside the data, show a clear message.

Group Consensus & Booking Mechanics

How does consensus and finalizing bookings work?

Members save preferred hotel options on the group page to visualize alignment. After discussing offline (or via chat), any member can click "Finalize Booking" for the group. Once locked, the booking cannot be edited, and the total cost is split equally per person.

What backend data and integrations are expected?

Catalog files must remain read-only, but store user profiles, posts, memberships, and mock bookings (generated IDs) in your database. Group memberships are fixed once joined, and dates follow the destination city's timezone per DATA_DICTIONARY.md. Real hotel APIs are not required.

How are pending approvals managed?

Display a notification badge on the group page for the creator to approve or reject pending join requests, persisting in the updated state.

Good luck!